

Research-Mind

One-Page Interview Explanation

Speak this as it is

Hello, my project is called **Research-Mind**. It is a multi-stage research application that helps students, researchers, and other users understand any topic using recent and traceable sources.

I started this project from a basic tutorial. The original flow had a Search Agent, a Reader Agent, a Writer Chain, and a Critic Chain. I kept the same simple structure because it is easy to understand and debug, but I improved every stage to make the project useful in a real situation.

The problem I wanted to solve is simple. A search engine gives many links, while a normal chatbot may give a good-looking answer without clear evidence. Research-Mind combines both ideas. It searches live sources, studies the collected information, writes a structured report, and then reviews that report.

The project works in four steps. First, the **Search Agent** understands the user's question and creates a short search plan. The application then collects recent academic papers from OpenAlex, news from Google News, and optional web results from Tavily. Second, the **Reader Agent** reads all the collected abstracts and snippets. It compares the sources and identifies important findings, recent signals, disagreements, and missing evidence. This is better than depending on only one webpage.

Third, the **Writer Chain** uses the Reader's notes and the original source records to create the final report. Every source receives an ID such as S1 or S2, and the Writer uses these IDs as citations. The prompt also tells the model not to invent sources or statistics and to mention uncertainty when the evidence is weak.

Fourth, the **Critic Chain** checks the completed report. It gives a score, explains the strengths, and points out unsupported claims, bias, missing evidence, or overconfidence.

Gemini is the main model for the Search, Reader, and Writer stages. Groq is used as an optional independent Critic. If Groq is not configured, Gemini also performs the review. I kept this rule fixed instead of creating a complicated model-routing system.

The frontend is built with Streamlit. The user can select the time range, research depth, language, region, audience, and sources. The application shows progress for all four stages and displays the report, Reader notes, sources, Critic feedback, and Search plan in separate tabs. The user can also download the report and ask follow-up questions.

One important limitation is that the system mainly uses abstracts and snippets. A valid source ID also does not guarantee that the source fully supports every claim. For high-stakes research, the user should open and verify the original sources. In the future, I would add automatic claim-citation verification and full-text support through official APIs.

Overall, this project shows how I converted a simple tutorial workflow into a practical research product while keeping the code readable, sequential, and easy to explain.